

1. COMIC CRAFT AI

AI Comic Story Creator using Gemini Models

ComicCraft is a web application that transforms simple story ideas into creative three-panel comic stories.

Project Presentation – 2026

2. TEAM MEMBERS

Team Members

1. K.Geethalakshmi
2. S.Gayathri
3. L. Arthi
4. S.Nivetha

Project: ComicCraft – The Magic Watch

3. INTRODUCTION & PROBLEM STATEMENT

Introduction

ComicCraft is an AI-powered web application that allows users to enter a simple story theme and generate a short three-panel comic story. It combines HTML, CSS, JavaScript, Flask and Gemini models.

Problem Statement

Creating a comic manually requires story planning, scene creation and dialogue writing. Beginners may find it difficult and time-consuming to convert a simple idea into a complete comic story.

4. PROPOSED SOLUTION & KEY FEATURES

Proposed Solution

Build an interactive web application where the user enters a theme, selects a story-generation option, and receives a three-panel comic. Gemini is used to generate the AI story.

Key Features

• Story theme input • Generate Comic • Create Story • New Story • Generate AI Story • Three comic panels • Gemini model integration • Error handling • Responsive styled interface

5. TECHNOLOGY STACK & SYSTEM WORKFLOW

Technology Stack

Frontend: HTML5, CSS3, JavaScript

Backend: Python Flask

AI: Google Gemini models / GenAI SDK

Development: Visual Studio Code

API Key: GEMINI_API_KEY environment variable

System Workflow

User enters theme → JavaScript sends POST request → Flask /generate route receives theme → Prompt is created → Gemini model generates 3 panels → Flask returns JSON → JavaScript displays panels in browser.

6. AI INTEGRATION & BENEFITS / APPLICATIONS

AI Integration

The JavaScript function `generateAIStory()` sends the selected theme to the Flask backend. The backend creates a structured prompt and calls the Gemini model. The response is split using '---' and displayed as three comic panels.

Benefits & Applications

- Faster story creation
- Supports creativity
- Beginner-friendly
- Interactive learning
- Entertainment
- Educational storytelling

YouTube/creative content prototypes

7. INDEX.HTML PROGRAM

Frontend: user interface, theme input, buttons, comic panels and AI request.

```
<!DOCTYPE html>
<html>
<head>
<title>ComicCraft - The Magic Watch</title>
<style>
* { box-sizing: border-box; }
body {
margin: 0; font-family: Arial, sans-serif;
background: linear-gradient(135deg,#f3e8ff,#e0c3fc);
color: #333;
}
header {
background:#6a1b9a; color:white; padding:35px 20px;
text-align:center;
}
header h1 { margin:0; font-size:42px; }
.container { max-width:900px; margin:30px auto; padding:20px; }
.creator-box {
background:white; padding:30px; border-radius:20px;
text-align:center; box-shadow:0 5px 20px #bbb;
}
input {
width:70%; padding:14px; border:2px solid #6a1b9a;
border-radius:10px; font-size:16px;
}
button {
padding:14px 22px; margin:8px 4px; border:none;
border-radius:10px; background:#6a1b9a;
color:white; font-size:16px; cursor:pointer;
}
.panel {
background:white; margin:20px auto; padding:25px;
max-width:650px; border-radius:18px;
border:3px solid #6a1b9a; text-align:center;
}
.panel-icon {
font-size:65px; background:#f3e8ff;
padding:20px; border-radius:15px;
}
.panel h3 { color:#6a1b9a; font-size:24px; }
.panel p { font-size:18px; line-height:1.6; }
</style>
</head>
<body>
<header>
<h1>ComicCraft</h1>
<p>AI Comic Story Creator using Gemini Models</p>
</header>

<div class="container">
<div class="creator-box">
<h2>Create Your Comic</h2>
<p>Enter your story idea below</p>
<input type="text" id="theme"
placeholder="Example: The Magic Watch">
<br>
<button onclick="generateComic()">Generate Comic</button>
<button onclick="createStory()">Create Story</button>
<button onclick="newStory()">New Story</button>
<button onclick="generateAISTory()">Generate AI Story</button>
</div>
<div id="story"></div>
</div>

<script>
function clearStory() {
```

```

    document.getElementById("story").innerHTML = "";
}

function generateComic() {
    clearStory();
    let theme = document.getElementById("theme").value ||
    "The Magic Watch";
    document.getElementById("story").innerHTML =
    "<div class='panel'><h3>Panel 1</h3><p>" +
    theme + ": A student discovers a mysterious magic watch.</p></div>" +
    "<div class='panel'><h3>Panel 2</h3><p>" +
    "The student turns the watch and time starts moving backwards.</p></div>" +
    "<div class='panel'><h3>Panel 3</h3><p>" +
    "The student uses the magic watch to solve a mysterious problem.</p></div>";
}

async function generateAIStory() {
    clearStory();
    const theme = document.getElementById("theme").value ||
    "The Magic Watch";

    try {
        const response = await fetch("/generate", {
            method:"POST",
            headers:{"Content-Type":"application/json"},
            body:JSON.stringify({theme:theme})
        });
        const data = await response.json();
        if (!response.ok) throw new Error(data.error || "Server error");

        let parts = data.story.split("---");
        let output = "<h2>AI Comic Story</h2>";
        for (let i=0;i<3;i++) {
            if(parts[i]) {
                let panelText = parts[i].trim()
                .replace(/^\s*\s*[123]\s*:\s*/i, "");
                output += "<div class='panel'><h3>Panel " +
                (i+1) + "</h3><p>" + panelText + "</p></div>";
            }
        }
        document.getElementById("story").innerHTML = output;
    } catch(error) {
        document.getElementById("story").innerHTML =
        "<div class='panel'><h3>AI Story Temporarily Unavailable</h3>" +
        "<p>Please try again later.</p></div>";
    }
}
</script>
</body>
</html>

```

8. SERVER.PY PROGRAM

Backend: Flask receives the theme, creates the Gemini prompt, generates the AI story and returns JSON.

```
from flask import Flask, jsonify, send_from_directory, request
from google import genai
import os

app = Flask(__name__)

# Gemini API
client = genai.Client(
    api_key=os.environ.get("GEMINI_API_KEY")
)

@app.route("/")
def home():
    return send_from_directory(".", "index.html")

@app.route("/generate", methods=["POST"])
def generate():
    try:
        data = request.get_json()
        theme = data.get("theme", "The Magic Watch")

        prompt = f"""
Create a simple 3-panel comic story about "{theme}".
The story must be about a student who finds a mysterious magic watch.

Panel 1:
The student finds the mysterious magic watch.

Panel 2:
The student turns the watch and time starts moving backwards.

Panel 3:
The student uses the magic watch to solve a mysterious problem.

Return exactly 3 panels.
Separate each panel using three hyphens:
Panel 1: ...
---
Panel 2: ...
---
Panel 3: ...

Keep the story short and simple.
Do not use markdown formatting.
"""

        response = client.models.generate_content(
            model="gemini-3.8-flash",
            contents=prompt
        )

        return jsonify({"story": response.text})

    except Exception as e:
        print("ERROR:", e)
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True)
```

9. OUTPUT - BROWSER VIEW

Browser View

The user sees the ComicCraft interface with a purple themed header, story input box and four action buttons.

Example: Enter The Magic Watch → click Generate AI Story → wait for Gemini response → three generated panels appear.

Output Panels

Panel 1 - Student finds the magic watch.

Panel 2 - Time starts moving backwards.

Panel 3 - Student uses the watch to solve the mysterious problem.

Browser flow: Input → JavaScript → Flask → Gemini → JSON → Three Comic Panels

10. CONCLUSION & FUTURE ENHANCEMENT

Conclusion

ComicCraft successfully combines web development and Gemini AI to transform a simple story theme into a three-panel comic story. The project demonstrates frontend interaction, backend API handling and AI-generated content.

Future Enhancements

- AI-generated comic images
- Character customization
- More panels
- Voice narration
- Download/share comic
- Multiple language support

Thank You